

ITAI 2377 Lab 04: Deep Learning Data Preprocessing

Instructor: [Your Name] **Date:** [Date]

Introduction

Welcome to Lab 04! We'll explore the critical role of data preprocessing in deep learning. Even though models can extract features, preprocessing is essential for optimal performance. We'll cover various data types and apply preprocessing techniques. Resources in Google Colab are limited, so efficient coding is key!

Why Preprocess?

Why preprocess when models extract features?

- **Standardization:** Models need consistent data formats and ranges.
- **Noise/Errors:** Raw data is messy. Preprocessing cleans it up.
- **Efficiency:** Cleaner data means faster training.
- **Results:** Good preprocessing helps models perform their best.

Think of preprocessing as a personal trainer for your model.

Data Types and Preprocessing Techniques

1. Image Data

```
import cv2
import numpy as np
from skimage import data, img_as_float
import matplotlib.pyplot as plt

# Load a sample image (replace with your image path if you have one)
image = data.camera() # Or use: image = cv2.imread("path/to/your/image.jpg")

# Resizing (Student Code: Resize the image to (128, 128))
# Hint: Use cv2.resize()
# YOUR CODE HERE
resized_image = cv2.resize(image, (128, 128))

# Color space conversion (to grayscale)
# The loaded 'image' from data.camera() is already grayscale (1 channel).
# cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) expects a 3-channel image.
# To fix this, we can directly use the 'image' as it's already grayscale.
gray_image = image.copy() # Using .copy() to ensure it's a separate object if needed

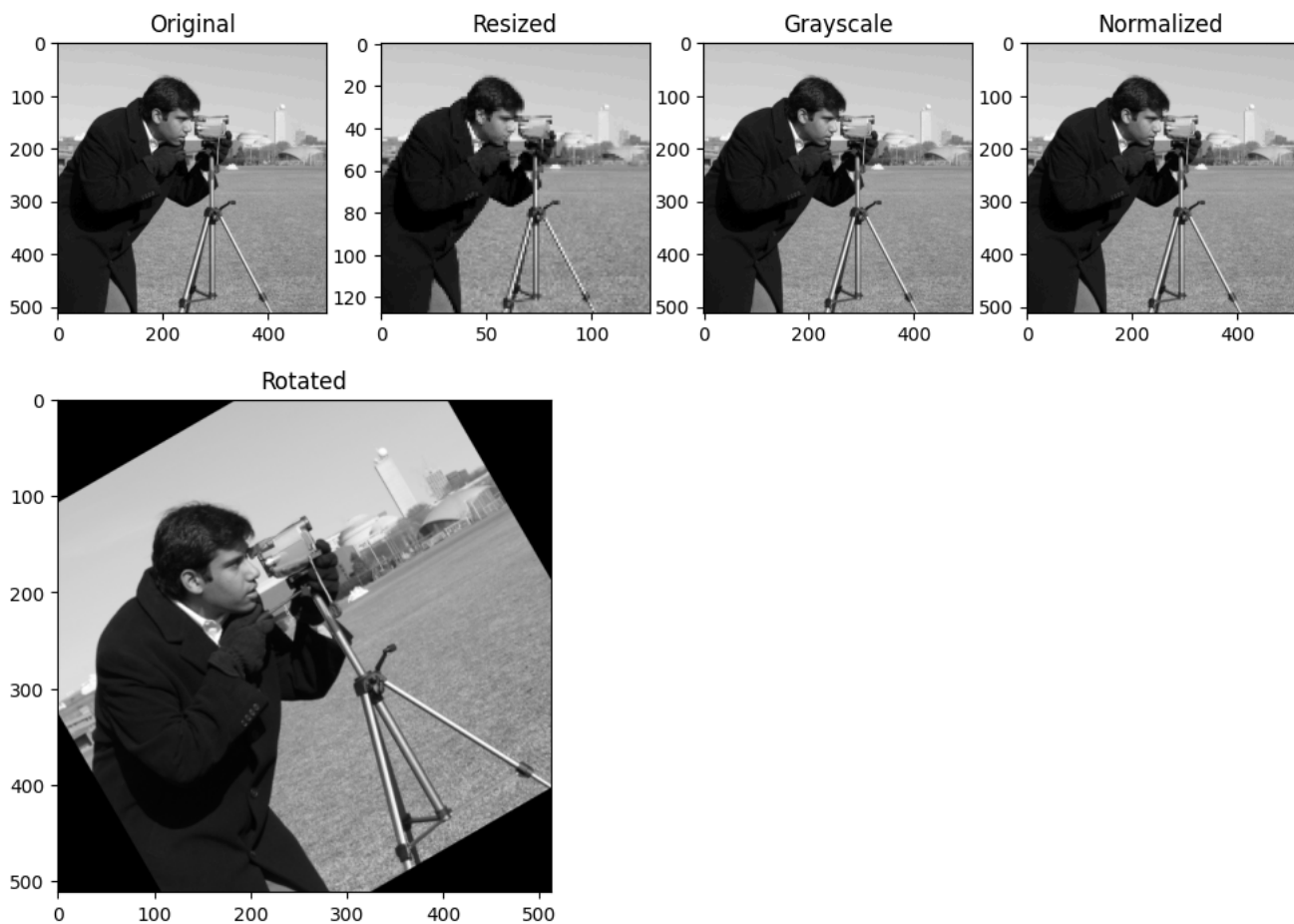
# Normalization (pixel values 0-1)
normalized_image = img_as_float(image)

# Display images
plt.figure(figsize=(12, 6))

plt.subplot(1, 4, 1), plt.imshow(image, cmap='gray'), plt.title("Original")
plt.subplot(1, 4, 2), plt.imshow(resized_image, cmap='gray'), plt.title("Resized")
plt.subplot(1, 4, 3), plt.imshow(gray_image, cmap='gray'), plt.title("Grayscale")
plt.subplot(1, 4, 4), plt.imshow(normalized_image, cmap='gray'), plt.title("Normalized")
plt.show()

# Data Augmentation (rotation - Student Code: Rotate by 30 degrees)
# Hint: Use cv2.getRotationMatrix2D and cv2.warpAffine
angle = 30 #YOUR CODE HERE
rows, cols = image.shape[:2]
M = cv2.getRotationMatrix2D((cols / 2, rows / 2), angle, 1)
rotated_image = cv2.warpAffine(image, M, (cols, rows))

plt.imshow(rotated_image, cmap='gray'), plt.title("Rotated")
plt.show()
```



2. Text Data

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import string

nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('punkt_tab', quiet=True) # Added to address the LookupError

text = "This is a fun example sentence with stop words and punctuation!"

# Tokenization (lowercase)
tokens = word_tokenize(text.lower())

# Stop word removal (Student Code: Remove stop words and punctuation)
# Hint: Use the 'stop_words' set and list comprehension
stop_words = set(stopwords.words('english'))
# YOUR CODE HERE
filtered_tokens = [w for w in tokens if not w in stop_words and w not in string.punctuation]

# Lemmatization (Student Code: Lemmatize the filtered tokens)
# Hint: Use WordNetLemmatizer()
lemmatizer = WordNetLemmatizer()
# YOUR CODE HERE
lemmatized_tokens = [lemmatizer.lemmatize(w) for w in filtered_tokens]

print("Original:", text)
print("Tokens:", tokens)
print("Filtered:", filtered_tokens)
print("Lemmatized:", lemmatized_tokens)

Original: This is a fun example sentence with stop words and punctuation!
Tokens: ['this', 'is', 'a', 'fun', 'example', 'sentence', 'with', 'stop', 'words', 'and', 'punctuation', '!']
Filtered: ['fun', 'example', 'sentence', 'stop', 'words', 'punctuation']
Lemmatized: ['fun', 'example', 'sentence', 'stop', 'word', 'punctuation']
```

3. Time Series Data

```
import pandas as pd
import numpy as np

# Sample data (with a missing value)
data = {'Date': pd.to_datetime(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05']),
        'Value': [10, 12, 15, np.nan, 18]}

df = pd.DataFrame(data)

# Missing value handling (Student Code: Use backward fill to fill missing values)
# Hint: Use fillna() with method='bfill'
# YOUR CODE HERE
df['Value'] = df['Value'].bfill() #modified after getting FutureWarning: A value is trying to be set on a copy of a DataFrame or
# Series through chained assignment using an inplace method. The behavior will change in pandas 3.0 and it suggested the new code.

# Normalization (min-max scaling - Student Code: Normalize the 'Value' column)
# YOUR CODE HERE
min_val = df['Value'].min()
max_val = df['Value'].max()
df['Normalized'] = (df['Value'] - min_val) / (max_val - min_val)

print(df)
```

	Date	Value	Normalized
0	2024-01-01	10.0	0.000
1	2024-01-02	12.0	0.250
2	2024-01-03	15.0	0.625
3	2024-01-04	18.0	1.000
4	2024-01-05	18.0	1.000

4. Optional: Video Data (Simplified)

```
import cv2
import matplotlib.pyplot as plt # Import matplotlib

# Load a video (replace with your video path or a small sample video if possible)
video_path = "Snake_on_the_trail.mp4" # Replace with your video file path (or upload one to Colab)
cap = cv2.VideoCapture(video_path)

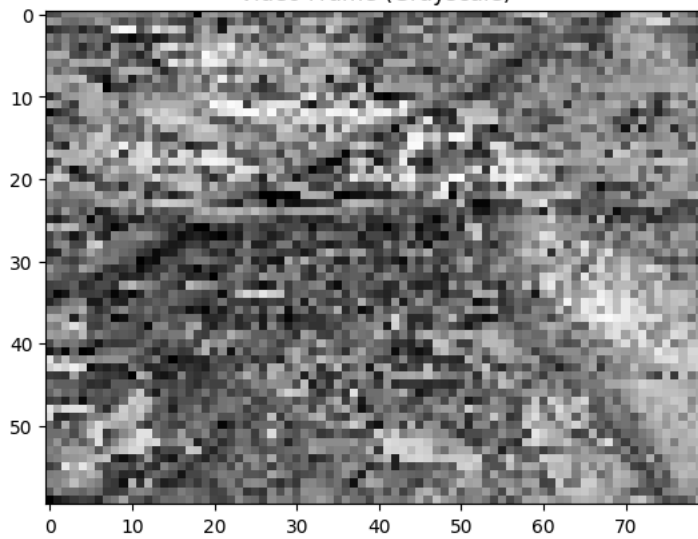
if not cap.isOpened():
    print("Error opening video")
else:
    ret, frame = cap.read() # Read a single frame
    if ret:
        # Resize the frame (for faster processing - Student Code: Resize to (80, 60))
        # YOUR CODE HERE
        resized_frame = cv2.resize(frame, (80, 60))

        # Convert to grayscale
        gray_frame = cv2.cvtColor(resized_frame, cv2.COLOR_BGR2GRAY) # OpenCV uses BGR

        # Display the frame (optional)
        plt.imshow(gray_frame, cmap='gray')
        plt.title("Video Frame (Grayscale)")
        plt.show()

    cap.release()
cv2.destroyAllWindows()
```

Video Frame (Grayscale)



5. Optional: Audio Data (Simplified)

```
import librosa
import numpy as np
import matplotlib.pyplot as plt

# Load an audio file (replace with your audio path)
audio_path = "Recording.wav" # Replace with your audio file path (or upload one to Colab)
y, sr = librosa.load(audio_path, duration=5) # Load a maximum of 5 seconds

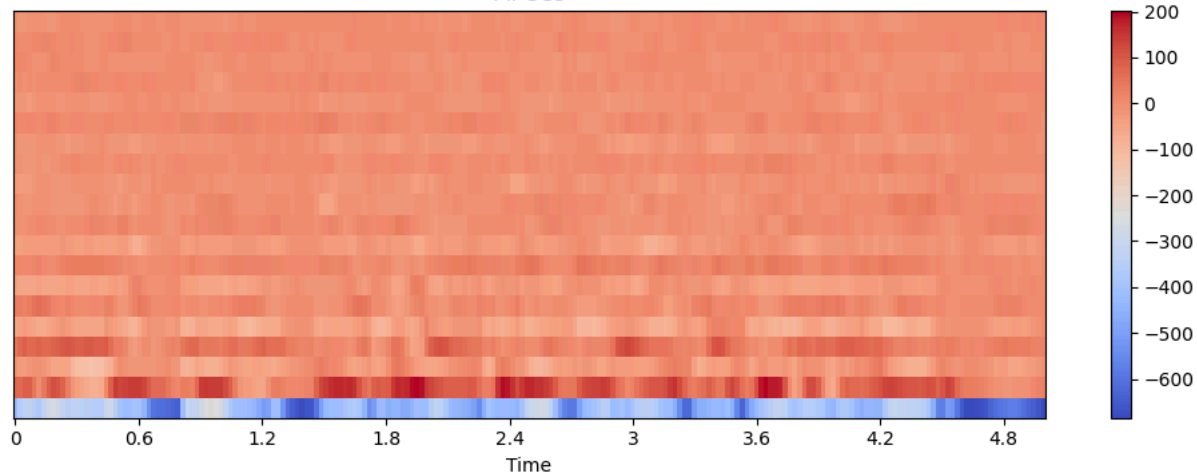
# Feature extraction (MFCCs - Student Code: Extract 20 MFCCs)
# Hint: Use librosa.feature.mfcc() with n_mfcc=20
# YOUR CODE HERE
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)

# Display MFCCs (optional)
plt.figure(figsize=(10, 4))
librosa.display.specshow(mfccs, sr=sr, x_axis='time')
plt.colorbar()
plt.title('MFCCs')
plt.tight_layout()
plt.show()

# Normalize MFCCs (example)
mfccs_normalized = (mfccs - np.mean(mfccs)) / np.std(mfccs)

print("MFCCs shape:", mfccs.shape)
print("Normalized MFCCs shape:", mfccs_normalized.shape)
```

MFCCs



MFCCs shape: (20, 216)
Normalized MFCCs shape: (20, 216)

Tip: Explore different methods for handling missing values (e.g., backward fill, interpolation). Consider feature engineering techniques like creating lagged variables.

Questions (Markdown Cell)

1. Why is data preprocessing still important even with deep learning's feature extraction capabilities?
2. Explain the difference between normalization and standardization. When would you choose one over the other?
3. Describe a scenario where data augmentation would be particularly useful.
4. What are some potential challenges or pitfalls to avoid during data preprocessing? How can you mitigate them?
5. Choose one of the data types covered in the lab (images, text, time series). Describe a specific real-world application that uses deep learning and explain how preprocessing would be crucial for that application.

Answers

1. Why preprocessing still matters Jumping between applications in images, text, audio, and time series, I kept coming back to the same thought: raw data is messy. There's formatting issues, missing values, background noise, and countless tiny aberrations models don't know how to deal with naturally. Preprocessing cleans those nuisances up. It normalizes things so there's less anxiety in the input data and the model can focus on what actually matters. You could even think of it as securing the path the model will run so it doesn't have to fight its way through the underbrush every time.
2. Normalization and standardization in practice Normalization and standardization both transform the range of your data, but for different reasons. Normalization scales your data into a bounded range, making it ideal if you already know the expected bounds of your original values. Standardization scales your data relative to its own mean and variance, which can be useful if your features vary in different ranges, or your model may benefit from centering around zero. Recently, I've started visualizing normalization as cleaning a house and standardization as moving the furniture around so the room is aesthetically pleasing. It just depends on how you want your model to "feel at home."
3. When augmentation becomes necessary To me, augmentation felt like hardening the model. When the true data distribution is chaotic - think different angles of an object, different accents and speeds of speech, etc..., augmentation allows you to model some of that variability. It can allow you to build a robust model with less data, or when it's impossible to collect more. Augmentation was like running your model through the Broadway pipeline with different lighting, costumes, and directions so it won't stop mid-performance if the scene changes ever so slightly.
4. Pitfalls you should avoid The beauty of preprocessing is that there are no hard rules. But this also means there are many opportunities for pitfalls. I've found it's easy to leak test data into training if you're not careful, apply transformations unevenly between train-test splits, or just go too far. I've oversanitized data by removing noise that was actually important, and augmented so much that my data no longer reflected reality. Whenever I work with a new dataset, I try to keep my eyes open and question as much as I can. Run tests to validate your assumptions early, and make your preprocessing steps explicit and repeatable.
5. A personal example where preprocessing played a key role Of all the data types I worked with, image data made the biggest impression on me. Thinking back to its applications in self-driving cars, it solidified my understanding of preprocessing. A car doesn't see the world already processed and it's unrealistic to expect your model to either. If your model doesn't natively run at video frame rates, you'll need to resize frames so they can be fed through the model quickly. You'll need to normalize pixel values or varying lighting conditions will disproportionately affect your model's predictions. You may even want to transform your images to better highlight the features you care about. Data augmentation was like exposing your model to rain, glare, shadows, and other events it will encounter on the road. Preprocessing bridges the gap between raw data and the idealized world your model expects to see.

Deliverables

- **Completed Notebook (PDF):** This notebook with your code, outputs, and answers to the questions.
- **Reflective Journal:** A short journal (1-2 pages) reflecting on your learning experience in this lab. Consider the following prompts:
 - What did you learn in this lab?
 - What challenges did you encounter? How did you overcome them?
 - Were there any concepts that you already knew?
 - Did anything surprise you?
 - What are some potential real-world applications of the preprocessing techniques you learned?